Data Visualization via Kernel Machines

III.10

Yuan-chin Ivan Chang, Yuh-Jye Lee, Hsing-Kuo Pao, Mei-Hsien Lee, Su-Yun Huang

10.1 Introduction 540
10.2 Kernel Machines in the Framework of an RKHS 541
10.3 Kernel Principal Component Analysis 543
Computation of KPCA 544
10.4 Kernel Canonical Correlation Analysis 551
10.5 Kernel Cluster Analysis 554

10.1 Introduction

Due to the rapid development of information technology in recent years, it is common to encounter enormous amounts of data collected from diverse sources. This has led to a great demand for innovative analytic tools that can handle the kinds of complex data sets that cannot be tackled using traditional statistical methods. Modern data visualization techniques face a similar situation and must also provide adequate solutions.

High dimensionality is always an obstacle to the success of data visualization. As well as this problem, explorations of the information and structures hidden in complicated data can be very challenging. Parametric models, on the one hand, are often inadequate for complicated data; on the other hand, traditional nonparametric methods can be far too complex to implement in a stable and affordable way due to the "curse of dimensionality." Thus, the development of new nonparametric methods for analyzing massive data sets is a highly demanding but important task. Following the recent successes in many fields of machine learning, kernel methods (e.g., Vapnik, 1995) can certainly provide us with powerful tools for such analyses. Kernel machines facilitate the flexible and versatile nonlinear analysis of data in a very high-dimensional (often with infinite dimensions) reproducing kernel Hilbert space (RKHS). The rich mathematical theory as well as topological and geometric structures associated with reproducing kernel Hilbert spaces enable probabilistic interpretation and statistical inference. They also provide a convenient environment that is suitable for massive computation.

In many classical approaches, statistical procedures are carried out directly on sample data in Euclidean space $\mathbb{R}^p$. In kernel methods, data are first mapped to a high-dimensional Hilbert space via a certain kernel or its spectrum, and classical statistical procedures are then applied to these kernel-transformed data. Kernel transformations provide us with a new way of specifying a "distance" or "similarity" metric between different elements.

After preparing the raw data in kernel form, standard statistical and/or mathematical software can be used to explore nonlinear data structures. For instance, we can perform nonlinear dimension reduction by kernel principal component analysis (KPCA), which can be used to construct high-quality classifiers as well as to provide new angles of view in data visualization. That is, we are able to view the more complicated (highly nonlinear) structures of massive data sets without the need to overcome the computational difficulties of building complex models. Many multivariate methods can also be extended to cover highly nonlinear cases through a kernel machine framework.

In this article, by combining the classical methods of multivariate analysis – such as PCA, canonical correlation analysis (CCA) and cluster analysis – with kernel machines, we introduce their kernelized counterparts, which enable more versatile and flexible data visualization.

Kernel Machines in the Framework of an RKHS

10.2

The goal of this section is twofold. First, it serves as an introduction to some basic RKHS theory that is relevant to kernel machines. Secondly, it provides a unified framework for kernelizing some classical linear methods, such as PCA, CCA, support vector clustering (SVC), etc., to allow for nonlinear structure exploration. For further details, we refer the reader to Aronszajn (1950) for the theory of reproducing kernels and reproducing kernel Hilbert spaces and Berlinet and Thomas-Agnan (2004) for their usage in probability, statistics and machine learning. Listed below are some definitions and basic properties.

- Let $\mathcal{X} \subset \mathbb{R}^p$ be the sample space of the data, which serves here as an index set. A real symmetric function $\kappa : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ is said to be positive definite if, for any positive integer m, any sequence of numbers $\{a_1, a_2, \dots, a_m \in \mathbb{R}\}$, and points $\{x_1, x_2, \dots, x_m \in \mathcal{X}\}$, we have $\sum_{i,j=1}^m a_i a_j \kappa(x_i, x_j) \geq 0$.
- An RKHS is a Hilbert space of real valued functions on $\mathcal{X}$ that satisfy the property that all evaluation functionals are bounded linear functionals. Note that an RKHS is a Hilbert space of pointwise-defined functions, where the $\mathcal{H}$-norm convergence implies pointwise convergence.
- For every positive definite kernel κ on $\mathcal{X} \times \mathcal{X}$ there is a corresponding unique RKHS, denoted by $\mathcal{H}_\kappa$, of real valued functions on $\mathcal{X}$. Conversely, for every RKHS $\mathcal{H}$ there is a unique positive definite kernel κ such that $\langle f(\cdot), \kappa(x, \cdot)\rangle_{\mathcal{H}} = f(x)$, $\forall f \in \mathcal{H}$, $\forall x \in \mathcal{X}$, which is known as the reproducing property. We say that this RKHS admits the kernel κ. A positive definite kernel is also termed a "reproducing kernel."
- A reproducing kernel κ that satisfies the condition $\int_{\mathcal{X}\times\mathcal{X}} \kappa^2(x, u)dxdu < \infty$ has a countable discrete spectrum given by

$$\kappa(x, u) = \sum_q \lambda_q \phi_q(x)\phi_q(u), \text{ or } \kappa := \sum_q \lambda_q \phi_q \otimes \phi_q \text{ for short}. \tag{10.1}$$

The main idea behind kernel machines is to first map the data into an Euclidean space $\mathcal{X} \subset \mathbb{R}^p$ into an infinite-dimensional Hilbert space. Next, a particular classical statistical procedure, such as PCA, is carried out in this feature Hilbert space. Such a hybrid model of a classical statistical procedure and a kernel machine is nonparametric in nature, but when fitting the data it uses the underlying parametric procedure (for example, the PCA finds some of the main linear components). The extra effort involved is the preparation of the kernel data before they are fed into some classical procedures. Below we will introduce two different but isomorphic maps that are used to embed the underlying Euclidean sample space into a feature Hilbert space. Consider the transformation

$$\Phi : x \mapsto (\sqrt{\lambda_1}\phi_1(x), \sqrt{\lambda_2}\phi_2(x), \dots, \sqrt{\lambda_q}\phi_q(x), \dots)'. \tag{10.2}$$

Let $\mathcal{Z} := \Phi(\mathcal{X})$, with is termed the *feature space*. The inner product in $\mathcal{Z}$ is given by

$$\Phi(x) \cdot \Phi(u) = \sum_q \lambda_q \phi_q(x) \phi_q(u) = \kappa(x, u) . \tag{10.3}$$

The kernel trick (10.3) of turning inner products in $\mathcal{Z}$ into kernel values allows us to carry out many linear methods in the spectrum-based feature space $\mathcal{Z}$ without needing to know the spectrum Φ itself explicitly. Therefore, it makes it possible to construct nonlinear (from the Euclidean space viewpoint) variants of linear methods. Consider another transformation,

$$\gamma : \mathcal{X} \mapsto \mathcal{H}_\kappa \text{ given by } \gamma(x) := \kappa(x, \cdot) , \tag{10.4}$$

which brings a point in $\mathcal{X}$ to an element in $\mathcal{H}_\kappa$. The original sample space $\mathcal{X}$ is thus embedded into a new sample space $\mathcal{H}_\kappa$. The map is called an Aronszajn map in Hein and Bousquet (2004). We connect these two maps (10.2) and (10.4) via $\mathcal{J} : \Phi(\mathcal{X}) \mapsto \gamma(\mathcal{X})$, given by $\mathcal{J}(\Phi(x)) = \kappa(x, \cdot)$. Note that $\mathcal{J}$ is a one-to-one linear transformation satisfying

$$\|\Phi(x)\|^2_{\mathcal{Z}} = \kappa(x, x) = \|\kappa(x, \cdot)\|^2_{\mathcal{H}_\kappa} = \|\gamma(x)\|^2_{\mathcal{H}_\kappa} .$$

Thus, $\Phi(\mathcal{X})$ and $\gamma(\mathcal{X})$ are isometrically isomorphic, and these two feature representations (10.2) and (10.4) are equivalent in this sense. Since they are equivalent, mathematically there is no distinction between them. However, from a data visualization perspective, there is a difference. As the feature map (10.2) is not explicitly known, there is no way of visualizing the feature data in $\mathcal{Z}$. In this article, for data visualization purposes, data or extracted data features are placed in the framework of $\mathcal{H}_\kappa$. We will use the feature map (10.4) for the later KPCA and kernel canonical correlation analysis (KCCA). Since the data cluster will be visualized in the original sample space $\mathcal{X}$ for the SVC, we will use the spectrum-based feature map (10.2) for ease.

Given data $\{x_1, \ldots, x_n\}$, let us write, for short, the corresponding new data in the feature space $\mathcal{H}_\kappa$ as

$$\gamma(x_j) := \gamma_j (\in \mathcal{H}_\kappa) . \tag{10.5}$$

As can be seen later, via these new data representations (10.4) and (10.5), statistical procedures can be solved in this kernel feature space $\mathcal{H}_\kappa$ in a parallel way using existing algorithms of classical procedures such as PCA and CCA. This is the key idea behind kernelization. The kernelization approach can be regarded, from the original sample space viewpoint, as a nonparametric method, since it adopts a model via kernel mixtures. It still has the computational advantage of keeping the process analogous to a parametric method, as its implementation only involves solving a parametric-like problem in $\mathcal{H}_\kappa$. The resulting kernel algorithms can be interpreted as running the original parametric (often linear) algorithms on kernel feature space $\mathcal{H}_\kappa$. For the KPCA and KCCA in this article, we use existing PCA and CCA codes on kernel data. One may choose to use codes from MATLAB, R, Splus or SAS. The extra programming effort involved is the preparation of data in an appropriate kernel form.

Let us discuss another computational issue. Given a particular training data set of size n, and by applying the kernel trick in (10.3), we can generate an $n \times n$ kernel data matrix according to a chosen kernel independent of the statistical algorithms to be used. We then apply the classical algorithm we are interested in, which depends only on the dot product, to the kernel matrix directly. Now the issue is that the generation of the full kernel matrix will become a stumbling block when the data size is huge due to the high computational cost (including CPU time and memory space). Moreover, the time complexity of the algorithm may depend on the size of this full kernel matrix. For example, the complexity of SVM is $O(n^3)$. To overcome these difficulties, Lee and Mangasarian (2001a) proposed the "reduced kernel" idea. Instead of using the full square kernel matrix $\mathbb{K}$, they randomly chose only a small portion of columns from $\mathbb{K}$ to form a thin rectangular kernel matrix, called a reduced kernel. The use of partial columns corresponds to the use of partial kernel bases in $\mathcal{H}_\kappa$, while all data points are used for model fitting when the full rows are retained. The idea of a reduced kernel is applied to the smooth support vector machine (Lee and Mangasarian, 2001a,b), and according to their numerical experiments, the reduced kernel method can dramatically reduce the computational load and memory usage without sacrificing much of the prediction accuracy. The heuristic is that the reduced kernel method regularizes the model complexity by reducing the number of kernel bases without sacrificing the number of data points that are used in model-fitting. This idea has also been successfully applied to smooth ϵ-insensitive support vector regression (Lee et al., 2005), which is a penalized ϵ-insensitive least squares fit that results in an adaptive ridge-type support vector regression estimator. For a theoretical study of the reduced kernel method, we refer the reader to Lee and Huang (2007). A comparison study of the empirical behavior of the eigenvalues and eigenvectors of the full kernel versus the reduced kernel can also be found therein. Also note that the feature representation (10.4) conveniently allows the related optimization problems to be solved in its primal form as opposed to the dual form. Primal optimization has several advantages which are especially prominent for large-scale problems. Primal optimization directly optimizes the objective function, the solution can often be obtained in only a couple of gradient steps, and it can easily accommodate the reduced kernel approach or another low-rank approximation approach for large-scale problems. While kernel machine packages are available and conveniently included in many software packages, such as R, Matlab, etc., this reduced kernel method allows us to utilize kernel machines with reduced computational effort, especially for large data sets. In this article, the reduced kernel is adopted in conjunction with the algorithms for KPCA and KCCA. Some reference papers and Matlab code are available at http://dmlab1.csie.ntust.edu.tw/downloads.

Kernel Principal Component Analysis 10.3

When dealing with high-dimensional data, methods of projection pursuit play a key role. Among various approaches, PCA is probably the most basic and commonly used

for dimension reduction. As unsupervised method, it looks for an r-dimensional linear subspace with $r < p$ that carries as much information (in terms of data variability) as possible. It sequentially finds new coordinate axes that provide the "best" ways to view the data, and along which the data exhibit the most variance. The set of all of the new coordinate axes, the so-called principal components, forms the basis set for the r-dimensional subspace. It is often the case that a small number of principal components are sufficient to account for most of the relevant data structure and information. These are sometimes called the factors or latent variables of the data. In classical PCA, we try to find the leading eigenvectors by solving an eigenvalue problem in the original sample space. We refer the reader to Mardia et al. (1979) and Alpaydin (2004) for further details. Due to its nature, PCA can only find linear structures in the data. But what if we are interested in both linear and nonlinear features? Inspired by the success of kernel machines, Schölkopf et al. (1998) and Schölkopf et al. (1999) raised the idea of kernel principal component analysis. In their papers, they apply the idea of PCA to the feature data in $\mathcal{Z}$ via the feature map (10.2). Their method allows the analysis of higher-order correlations between input variables. In practice, the transformation does not need to be specified explicitly, and the whole operation can be done by computing the dot products in (10.3). In this chapter, our formulation of KPCA is constructed in terms of its equivalent variant in the framework of RKHS $\mathcal{H}_\kappa$ given by (10.4).

Actually, given any algorithm that can be expressed solely in terms of dot products (i.e., without explicit usage of the variables $\Phi(x)$ themselves), the kernel method enables us to create nonlinear versions of thr given algorithm; see Aizerman et al. (1964) and Boser et al. (1992) for example. This general fact is well known to the machine learning community, and is gradually gaining popularity in the statistical community too. Here we give some examples of the application of this method to the domain of unsupervised learning, in order to obtain a nonlinear form of PCA. Some data sets from UCI Machine Learning Benchmark data archives are used for illustration.

10.3.1 Computation of KPCA

Before getting into KPCA, we briefly review the computational procedure of classical PCA. Let $X \in \mathbb{R}^p$ be a random vector with covariance matrix $\Sigma := \mathrm{Cov}(X)$. To find the first principal component, we must find a unit vector $w \in \mathbb{R}^p$ such that the variance of the projection of X along w is maximized, i.e.,

$$\max_{w} \; w'\Sigma w \text{ subject to } \|w\| = 1\,. \tag{10.6}$$

This can be rewritten as a Lagrangian problem:

$$\max_{\alpha, w} \; w'\Sigma w - \alpha(w'w - 1)\,, \tag{10.7}$$

where α is the Lagrange multiplier. Taking derivatives with respect to α and w and setting them to zero, we then solve for α and w. The solutions are denoted α_1 and w_1,

and they must satisfy $\Sigma w_1 = \alpha_1 w_1$, and $w_1{}' w_1 = 1$. Therefore, w_1 is obtained by finding the eigenvector associated with the leading eigenvalue α_1. For the second principal component, we look for a unit vector w_2 which is orthogonal to w_1 and maximizes the variance of the projection of X along w_2. That is, in terms of a Lagrangian problem, we solve for α_2, w_2 and β in the following optimization formula:

$$\max_{\alpha_2,\beta,w_2} w_2'\Sigma w_2 - \alpha_2(w_2'w_2 - 1) - \beta(w_1'w_2)\,. \tag{10.8}$$

Using a similar procedure, we are able to find the leading principal components sequentially.

Assume for simplicity that the data $\{x_1,\ldots,x_n\}$ are already centered on their mean, and so the sample covariance matrix is given by $\Sigma_n = \sum_{j=1}^n x_j x_j'/n$. By applying the above sequential procedure to the sample covariance Σ_n, we can obtain the empirical principal components.

For KPCA using the feature representation (10.4), the data mapped in the feature space $\mathcal{H}_\kappa$ are $\{\gamma_1,\ldots,\gamma_n\}$. The sample covariance (which is also known as a covariance operator in $\mathcal{H}_\kappa$) is given by

$$C_n := \frac{1}{n}\sum_{j=1}^n (\gamma_j - \bar{\gamma}) \otimes (\gamma_j - \bar{\gamma})\,, \tag{10.9}$$

where $f \otimes g$ is a linear operator defined by $(f \otimes g)(h) := \langle g, h\rangle_{\mathcal{H}_\kappa} f$ for $f, g, h \in \mathcal{H}_\kappa$. Applying similar arguments to before, we aim to find the leading eigencomponents of C_n. That is, we solve for h in the following optimization problem:

$$\max_{h\in\mathcal{H}_\kappa} \langle h, C_n h\rangle_{\mathcal{H}_\kappa} \text{ subject to } \|h\|_{\mathcal{H}_\kappa} = 1\,. \tag{10.10}$$

It can be shown that the solution to this is of the form $h = \sum_{j=1}^n \beta_j \gamma_j \in \mathcal{H}_\kappa$, where β_j's are scalars. As

$$\langle h, C_n h\rangle_{\mathcal{H}_\kappa} = \sum_{i,j=1}^n \beta_i\beta_j\langle \gamma_i, C_n\gamma_j\rangle_{\mathcal{H}_\kappa} = \beta'\mathbb{K}\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta/n\,,$$

and $\|h\|^2_{\mathcal{H}_\kappa} = \beta'\mathbb{K}\beta$, where $\mathbb{K} = \left[\kappa(x_i, x_j)\right]$ denotes the $n \times n$ kernel data matrix, the optimization problem can be reformulated as

$$\max_{\beta\in\mathbb{R}^n} \beta'\mathbb{K}\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta/n \text{ subject to } \beta'\mathbb{K}\beta = 1\,. \tag{10.11}$$

The Lagrangian of the above optimization problem is

$$\max_{\alpha\in\mathbb{R},\beta\in\mathbb{R}^n} \beta'\mathbb{K}\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta/n - \alpha(\beta'\mathbb{K}\beta - 1)\,,$$

where α is the Lagrange multiplier. Taking derivatives with respect to the β's and setting them to zero, we get

$$\mathbb{K}\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta/n = \alpha\mathbb{K}\beta\,, \text{ or } \left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta = n\alpha\beta\,. \tag{10.12}$$

This leads to the eigenvalues–eigenvectors problem for $\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}$. Denoting its largest eigenvalue by α_1 (note that the multiplicative factor n is absorbed into the eigenvalue) and its associated eigenvector by β_1, then the corresponding first kernel principal component is given by $h_1 = \sum_{j=1}^{n} \beta_{1j}\gamma_j$ in the feature space $\mathcal{H}_\kappa$. We can then sequentially find the second, third, etc., principal components. From (10.12), we have that β_k, $k = 1, 2, \ldots$, are orthogonal to $\mathbf{1}_n$ and so the normalization $\beta_k'\mathbb{K}\beta_k = 1$ is equivalent to $\beta_k'\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}\beta_k = 1$. Thus, β_k is normalized according to $\alpha_k\beta_k'\beta_k = 1$.

For an $x \in \mathbb{R}^p$ and its feature image $\gamma(x) \in \mathcal{H}_\kappa$, the projection of $\gamma(x)$ along the kth eigencomponent of C_n is given by

$$\langle \gamma(x), h_k \rangle_{\mathcal{H}_\kappa} = \langle \gamma(x), \sum_{j=1}^{n} \beta_{kj}\gamma_j \rangle_{\mathcal{H}_\kappa} = \sum_{j=1}^{n} \beta_{kj}\kappa(x_j, x), \tag{10.13}$$

where β_k is the kth eigenvector of $\left(I_n - \frac{\mathbf{1}_n\mathbf{1}_n'}{n}\right)\mathbb{K}$. Therefore, the projection of $\gamma(x)$ onto the dimension reduction linear subspace spanned by the leading r eigencomponents of C_n is given by

$$\left(\sum_{j=1}^{n} \beta_{1j}\kappa(x_j, x), \ldots, \sum_{j=1}^{n} \beta_{rj}\kappa(x_j, x)\right) \in \mathbb{R}^r .$$

Let us demonstrate the idea behind KPCA using a few examples. There are three data sets in this demonstration, the synthesized "two moon" data set, the "Pima diabetes" data set, and the "image segmentation" data set.

5 **Example 5** First, we compare PCA and KPCA using the synthesized "two moons" data set shown in Fig. 10.1. The original data set is located in a 2-D space in (a). We can see that the two classes of data are not well-separated along any one-dimensional component. Therefore, upon applying PCA, we are not going to see good separation along the first principal coordinate axis. In the histogram (b1), the horizontal axis is the first principal coordinate from the PCA and the vertical axis is the frequency. As we can see, there is a great deal of overlap between two classes. A kernelized projection can provide a solution to this problem. In the histogram (b2), the horizontal axis is the first principal coordinate given by KPCA (with the polynomial kernel of degree 3) and the vertical axis is the frequency. Note that the KPCA with polynomial kernel also does not give good separation. However, in the histogram (b3), the results obtained from KPCA using a radial basis function (RBF, also known as the "Gaussian kernel") with $\sigma = 1$ are shown, and they exhibit a good separation. If either PCA or KPCA is to be used as a preprocessing step before a classification task, clearly KPCA using the radial basis function with $\sigma = 1$ is the best choice. The ROC curves for these approaches are shown in (c), with the area under the curve (AUC) reported as $\mathcal{A}_{\text{PCA}} = 0.77$, $\mathcal{A}_{\text{KPCA(Poly)}} = 0.76$ and $\mathcal{A}_{\text{KPCA(RBF)}} = 0.91$. KPCA with RBF ($\sigma = 1$) is clearly better at distinguishing between two groups than classical PCA and the KPCA using the polynomial kernel.

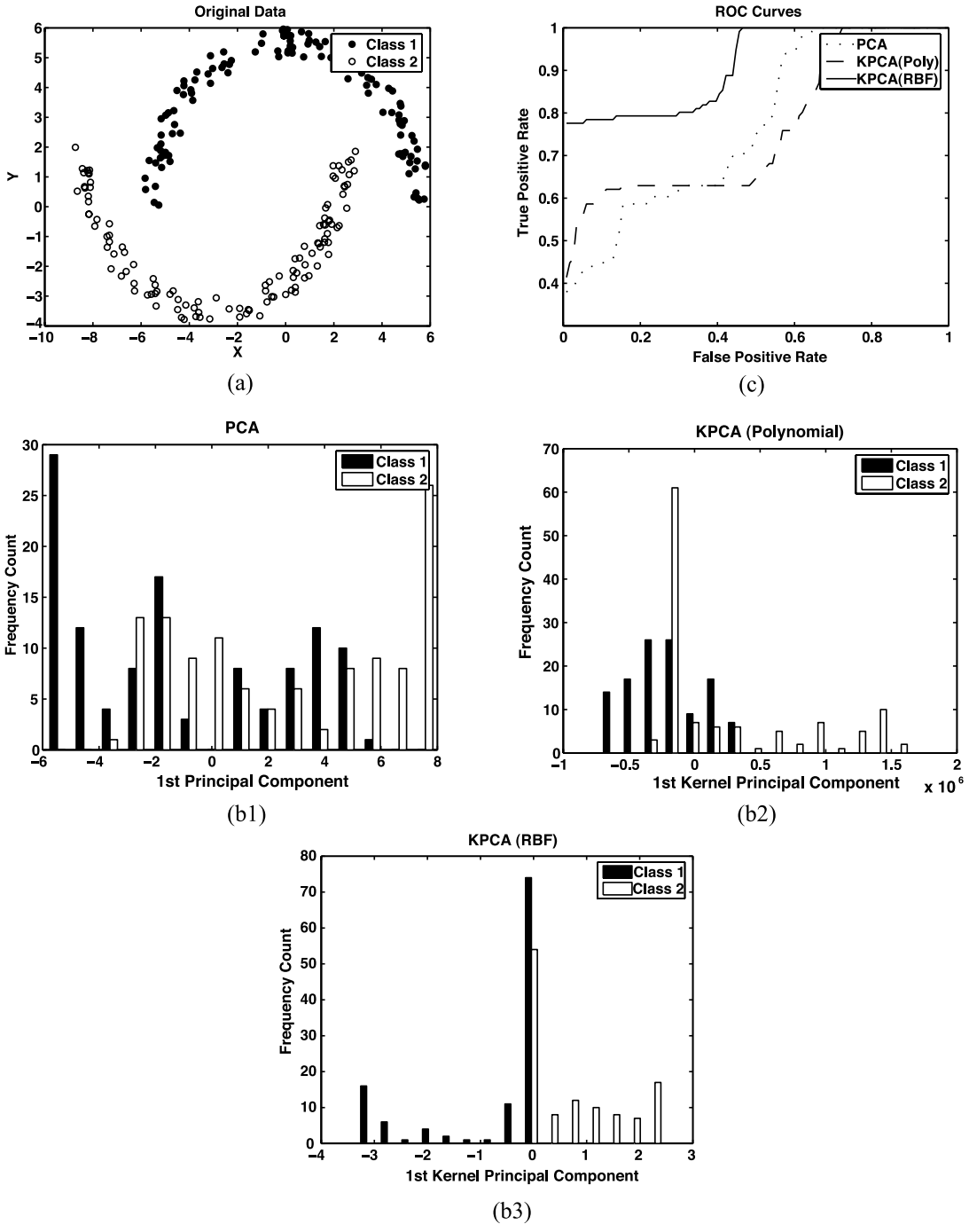


Figure 10.1. PCA and KPCA applied to a synthetic "two moons" data set: (*a*) original data, (*b1*) PCA results, (*b2*) results from KPCA with the polynomial kernel of degree 3, (*b3*) results from KPCA with the Gaussian kernel, $\sigma = 1$. In each of these histograms (*b1–b3*), the horizontal axis is the principal coordinate from either the PCA or KPCA and the vertical axis is the frequency. Plot (*c*) shows the ROC curves for these three procedures, with area under curve reported as $\mathcal{A}_{\mathrm{PCA}} = 0.77$, $\mathcal{A}_{\mathrm{KPCA(Poly)}} = 0.76$ and $\mathcal{A}_{\mathrm{KPCA(RBF)}} = 0.91$

6 **Example 6** In this example we use the "Pima diabetes" data set from the UCI Machine Learning data archives. The reduced kernel method is adopted by randomly sampling 20% of the column vectors from the full kernel matrix. For reduced kernel PCA, we assume $\tilde{\mathbb{K}}$ is the underlying reduced kernel data matrix of size $n \times m$, where n is the data size and m is the reduced set size (i.e., column set size). KPCA using the reduced kernel is a singular value decomposition problem that involves extracting the leading right and left singular vectors $\tilde{\beta}$ and β:

$$\left(I_n - \frac{\mathbf{1}_n \mathbf{1}_n'}{n}\right) \tilde{\mathbb{K}} \tilde{\beta} = \alpha\beta\,, \text{ normalized to } \alpha\tilde{\beta}'\tilde{\beta} = 1 \text{ and } \alpha\beta'\beta = 1\,. \tag{10.14}$$

In this data set, there are nine variables, including number of times pregnant, plasma glucose concentration (glucose tolerance test), diastolic blood pressure (mm Hg), triceps skin fold thickness (mm), two-hour serum insulin (mu U/ml), body mass index (weight in kg/(height in m)^2), diabetes pedigree function, age (years), and the class variable (test for diabetes), which can be positive or negative. For demonstration purposes, we use the first eight variables as input measurements and the last variable as

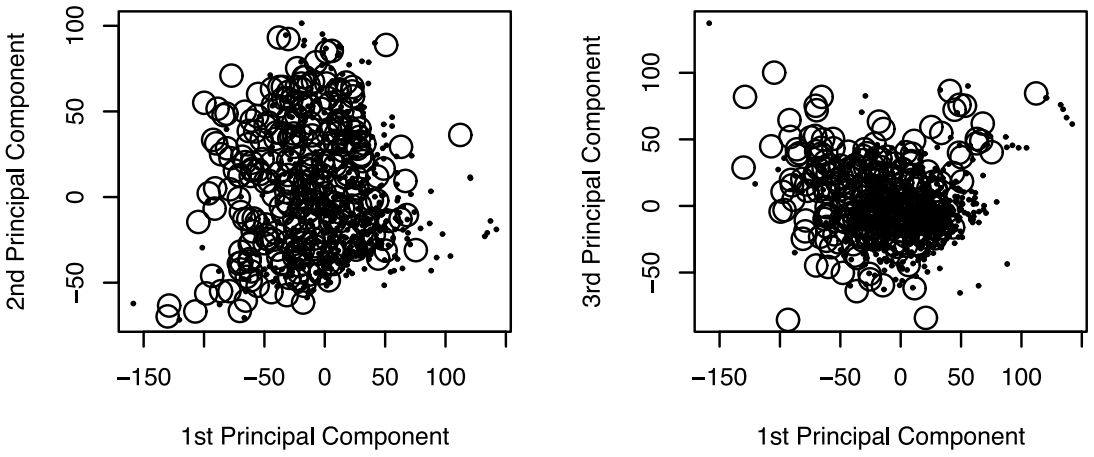


Figure 10.2. Results from PCA based on original input variables

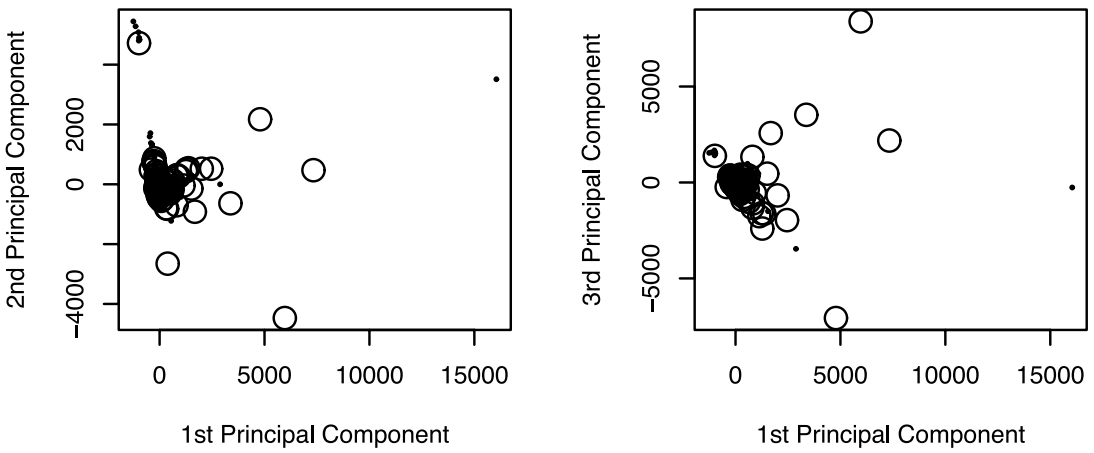
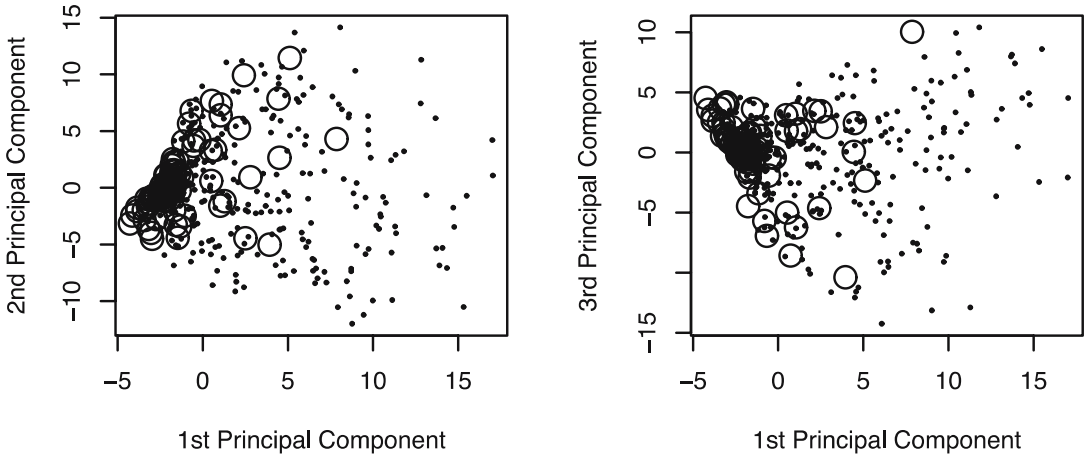
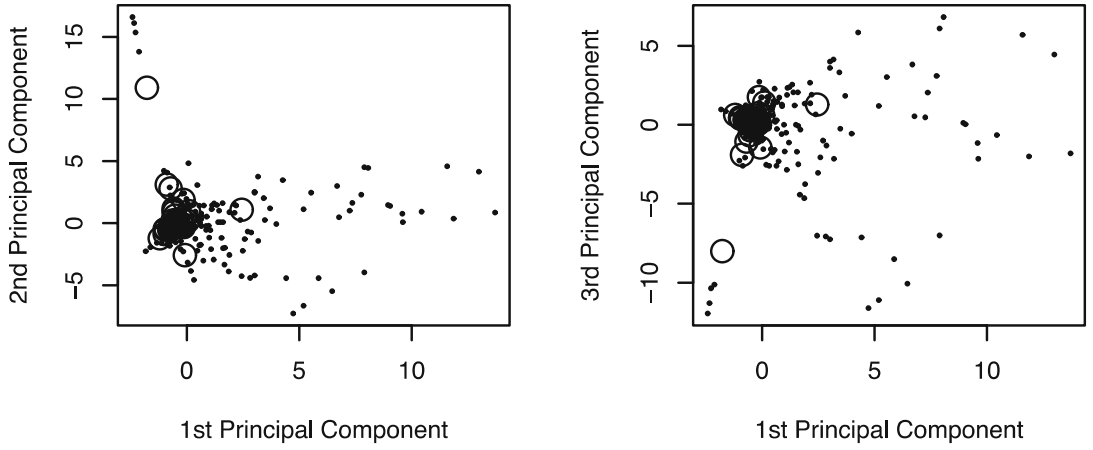


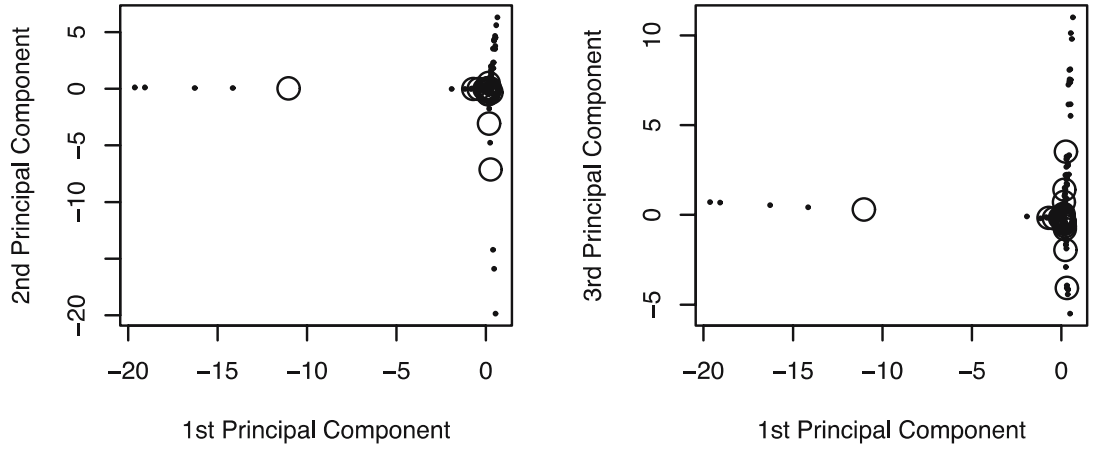
Figure 10.3. Results from KPCA with the polynomial kernel of degree 3 and scale 1



(a) $\sigma^2 = 1/2$



(b) $\sigma^2 = 1/6$



(c) $\sigma^2 = 1/10$

Figure 10.4. Results from KPCA with Gaussian kernels

the class variable. PCA and KPCA using both the polynomial kernel and the Gaussian kernel are carried out on all of the input measurements. In Figs. 10.2–10.4, the circles and dots denote positive and negative samples, respectively. Figure 10.2 shows the data scatter projected onto the subspace spanned by the first three principal components produced by the classical PCA. Similarly, Fig. 10.3 shows plots of data scatter projected onto the subspace spanned by the first three principal components obtained by KPCA using a polynomial kernel of degree 3 and scale parameter 1. Figures 10.4a–c are pictures of projections onto the principal component space produced by Gaussian kernels with $\sigma^2 = 1/2, 1/6$ and $1/10$, respectively. If we compare Fig. 10.2 (obtained using PCA) with the other plots, it is clear that KPCA provides some extra information about the data that cannot be obtained through classical PCA.

7 **Example 7** In the third series (Fig. 10.5), we apply PCA KPCA to the "image segmentation" data set (also from the UCI Machine Learning data archives), which consists of 210 data points (each with 19 attributes) that are classified into seven classes. KPCA with the RBF gives better class separation than PCA, as can be seen in (a1) & (b1). When all seven classes are plotted in one graph, it is hard to determine the effect of

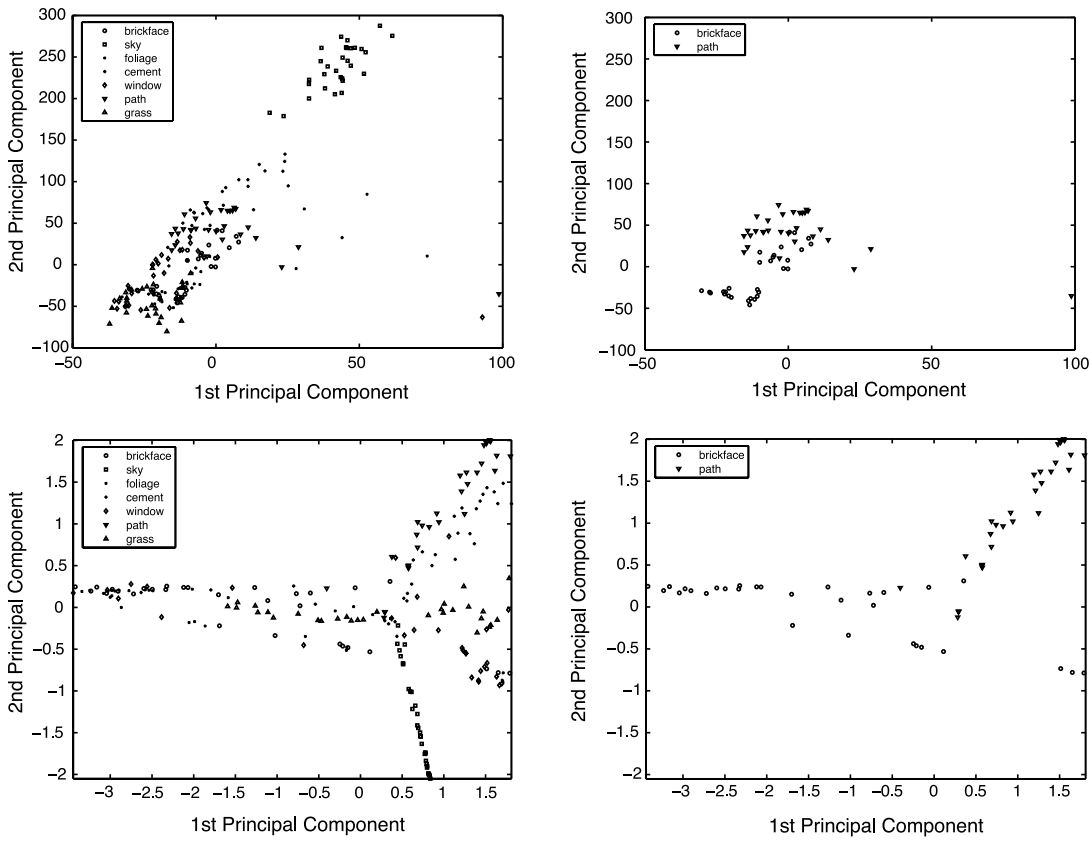


Figure 10.5. Results from applying PCA and KPCA to the "image segmentation" data set. A limited number of outliers have been omitted in the figures

KPCA. Therefore, we have also provided figures retaining only "brickface" and "path." These clearly show that separation is produced by KPCA but not by the PCA.

Remark 1 The selection of the kernel and its window width is still an issue in general kernel methodology. There are some works that provide guidance on kernel selection for classification and supervised learning problems, but there is still a lack of guidelines in clustering and unsupervised learning problems. In this work, we merely aimed to show that nonlinear information about data can be obtained by applying kernel methods with only minor effort required. The kernel method can help to dig out nonlinear information about the data, which would be difficult or impossible to obtain by applying classical linear PCA in the original input space. **1**

Kernel Canonical Correlation Analysis **10.4**

Researchers have long been interested in describing and classifying relations between two sets of variables. Hotelling (1936) introduced canonical correlation analysis to describe the linear relation between two sets of variables that have a joint distribution. This defines a new coordinate system for each of the sets such that the new pair of coordinate systems are the best at maximizing correlations. The new systems of coordinates are simply linear systems of the original ones. Thus, classical CCA can only be used to describe linear relations. Using such linear relations, classical CCA can only find linear dimension reduction subspace and linear discriminant subspace. However, motivated by the active development and the popular and successful usage of various kernel machines, a hybrid approach combining classical CCA with a kernel machine (Akaho, 2001; Bach and Jordan, 2002), named *kernel canonical correlation analysis*, has emerged in recent years. KCCA has also been studied recently by Hardoon et al. (2004) among others.

Suppose a random vector X with p components has a probability distribution P on $\mathcal{X} \subset \mathbb{R}^p$. We partition X into

$$X = \begin{bmatrix} X^{(1)} \\ \mathcal{X}^{(2)} \end{bmatrix},$$

with p_1 and p_2 components, respectively. The corresponding partition of $\mathcal{X}$ is denoted by $\mathcal{X}_1 \oplus \mathcal{X}_2$. We are interested in finding relations between $X^{(1)}$ and $X^{(2)}$. Classical CCA is concerned with linear relations. It describes linear relations by reducing the correlation structure between these two sets of variables to the simplest possible form by means of linear transformations of $X^{(1)}$ and $X^{(2)}$. It finds pairs $(\alpha_i, \beta_i) \in \mathbb{R}^{p_1+p_2}$ in the following way. The first pair maximizes the correlation between $\alpha_1' X^{(1)}$

and $\beta_1' X^{(2)}$ subject to the unit variance constraints $\mathcal{V}(\alpha_1' X^{(1)}) = \mathcal{V}(\beta_1' X^{(2)}) = 1$, and the kth pair (α_k, β_k), which is uncorrelated with the first $k-1$ pairs, maximizes the correlation between $\alpha_k' X^{(1)}$ and $\beta_k' X^{(2)}$, and is again subject to the unit variance constraints. The sequence of correlations between $\alpha_i' X^{(1)}$ and $\beta_i' X^{(2)}$ describes only the linear relations between $X^{(1)}$ and $X^{(2)}$. There are cases where linear correlations may not be adequate for describing the "associations" between $X^{(1)}$ and $X^{(2)}$. A natural alternative, therefore, is to look for nonlinear relations. Kernel methods can provide a convenient approach to nonlinear generalization. Let $\kappa_1(\cdot,\cdot)$ and $\kappa_2(\cdot,\cdot)$ be two positive definite kernels defined on $\mathcal{X}_1 \times \mathcal{X}_1$ and $\mathcal{X}_2 \times \mathcal{X}_2$, respectively. Let $\mathbb{X}$ denote the data matrix given by

$$\mathbb{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}_{n \times p} .$$

Each data point (as a row vector) $x_j = (x_j^{(1)}, x_j^{(2)})$ in the data matrix is transformed into a kernel representation:

$$x_j \mapsto y_j = (y_j^{(1)}, y_j^{(2)}), \tag{10.15}$$

where

$$y_j^{(i)} = (\kappa_i(x_j^{(i)}, x_1^{(i)}), \ldots, \kappa_i(x_j^{(i)}, x_n^{(i)})), \; j = 1, \ldots, n, \text{ and } i = 1, 2 .$$

Or, in matrix notation, the kernel data matrix is given by

$$\mathbb{K} = [\mathbb{K}_1 \mathbb{K}_2] = \begin{bmatrix} y_1^{(1)} & y_1^{(2)} \\ \vdots & \vdots \\ y_n^{(1)} & y_n^{(2)} \end{bmatrix}_{n \times 2n} , \tag{10.16}$$

where $\mathbb{K}_i = [\kappa_i(x_j^{(i)}, x_{j'}^{(i)})]_{j,j'=1}^n$, $i = 1, 2$, are the full kernel matrices for data $\{x_j^{(i)}\}_{j=1}^n$. The representation of x_j by $y_j = (y_j^{(1)}, y_j^{(2)}) \in \mathbb{R}^{2n}$ can be regarded as an alternative way of recording data measurements with high inputs.

The KCCA procedure consists of two major steps:

(a) Transform the data points into a kernel representation, as in (10.15) or (10.16), in matrix notation.

(b) Apply the classical CCA procedure to the kernel data matrix $\mathbb{K}$. Note that some kind of regularization is needed here to solve the associated spectral problem of extracting leading canonical variates and correlation coefficients. Here we use the reduced kernel concept stated in the RKHS section and in Example 6. Only partial columns are computed to form reduced kernel matrices, denoted by $\tilde{\mathbb{K}}_1$ and $\tilde{\mathbb{K}}_2$. The classical CCA procedure is applied to the reduced kernel matrix $[\tilde{\mathbb{K}}_1 \; \tilde{\mathbb{K}}_2]$.

As KCCA is simply the application of classical CCA to kernel data, existing code from standard statistical packages can be utilized. In the example below we use the MATLAB m-file "canoncorr" to implement classical CCA on kernel data.

Example 8 We use the data set "pen-based recognition of hand-written digits" from the UCI Machine Learning data archives for a visual demonstration of nonlinear discriminant using KCCA. We use the 7494 training instances for explanatory purposes. For each instance, there are 16 input measurements (i.e., x_j is 16-dimensional) and a corresponding group label y_j from $\{0,1,2,\ldots,9\}$. A Gaussian kernel with a window width $(\sqrt{10S_1},\ldots,\sqrt{10S_{16}})$ is used to prepare the kernel data, where S_i's are the coordinate-wise sample covariances. A reduced kernel of size 300 equally stratified over ten digit groups is used and serves as the $\tilde{\mathbb{K}}_1$ in step (b) of the KCCA procedure. We use y_j, the group labels, as our $\mathbb{K}_2$ (no kernel transformation involved). Precisely, 8

$$\mathbb{K}_2 = \begin{bmatrix} Y_1' \\ \vdots \\ Y_n' \end{bmatrix}_{n\times 10}, \qquad Y_j' = (0,\ldots,1,0,\ldots),$$

where Y_j is a dummy variable for group membership. If $y_j = i$, $i = 0,1,\ldots,9$, then Y_j has the entry 1 at the $(i+1)$th position and 0 elsewhere. Now we want to search for relations between the input measurements and their associated group labels using CCA and KCCA. The training data are used to find the leading CCA- and KCCA-derived variates. Next, 20 test samples from each digit group are drawn randomly from the test set [1]. Scatter plots of test data projected along the leading CCA-derived variates

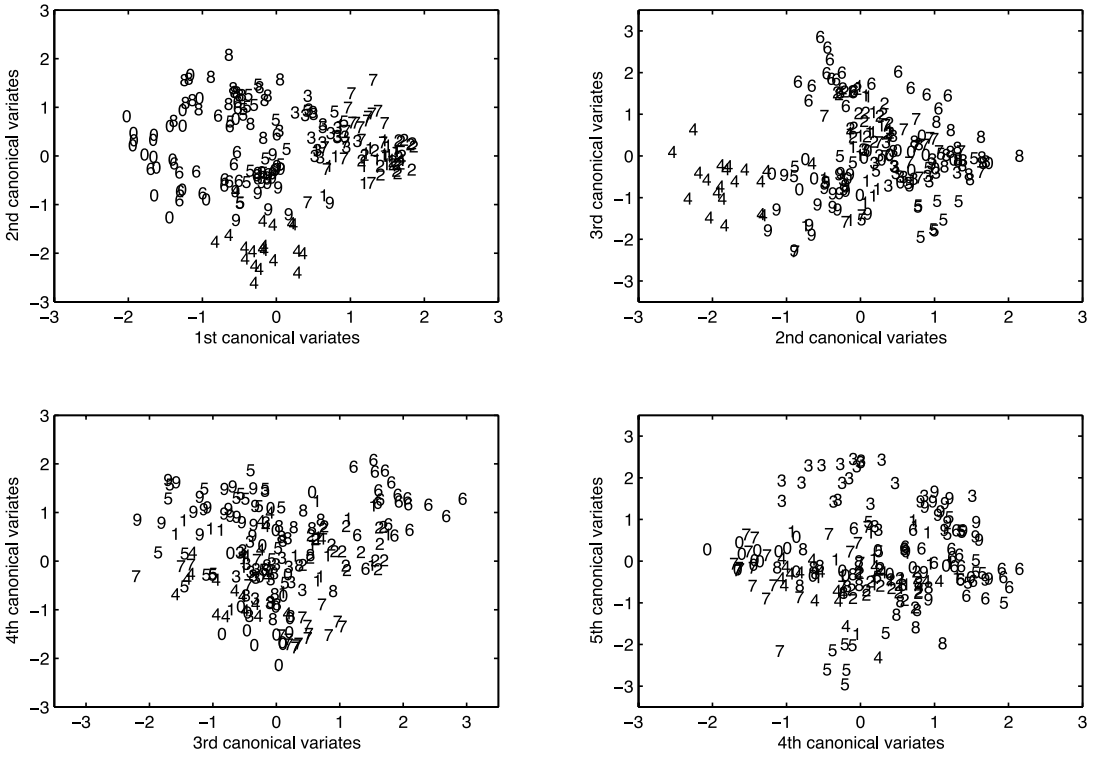


Figure 10.6. Scatter plots of pen digits over CCA-derived variates

[1] The test set has 3498 instances in total, with around 350 instances on average for each digit. For the sake of plot clarity and to avoid excess ink, we use only 20 test points per digit.

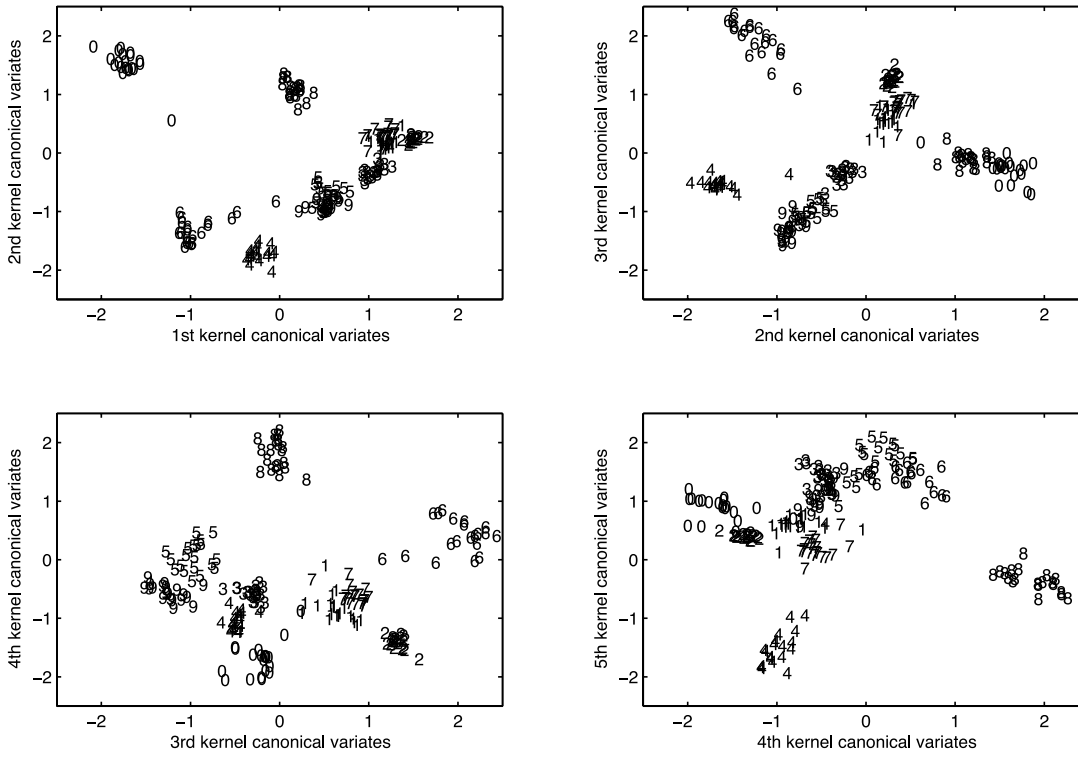


Figure 10.7. Scatter plots of pen digits over KCCA-derived variates

(Fig. 10.6) and the leading KCCA-derived variates (Fig. 10.7) are given below. Different groups are labeled with different digits. It is clear that the CCA-derived variates are not informative regarding group labels, while the KCCA-derived variates are.

10.5 Kernel Cluster Analysis

Cluster analysis is categorized as an unsupervised learning method, which tries to find the group structure in an unlabeled data set. A cluster is a collection of data points which are "similar" to points in the same cluster, according to certain criteria, and are "dissimilar" to points belonging to other clusters. The simplest clustering method is probably the k-means algorithm (which can be used in a hybrid approach with a kernel machine, or as a standalone method). Given a predetermined number of clusters k, the k-means algorithm will proceed to group data points into k clusters by (1) placing k initial centroids in the space, (2) assigning each data point to the cluster of its closest centroid, (3) updating the centroid positions and repeat the steps (1) and (2) until some stopping criterion is reached (see MacQueen, 1967). Despite its simplicity, the k-means algorithm does have some disadvantages. First, a predetermined k is necessary for the algorithm input, and different k's can lead to dramatically different results. Secondly, suboptimal results can occur for certain

initial choices of centroid seeds. Thirdly, the algorithm may not be appropriate for some data distributions where the metric is not uniformly defined, i.e., the idea of "distance" has different meanings in different regions or for data belonging to different labels.

Here we address these issues by introducing a different clustering approach, namely *support vector clustering*, which allows hierarchical clusters with versatile clustering boundaries. Below we briefly describe the idea behind SVC, as described by Ben-Hur et al. (2001). SVC was inspired by support vector machines and kernel methods. In SVC, data points are mapped from the data space $\mathcal{X}$ to a high-dimensional feature space $\mathcal{Z}$ by a nonlinear transformation (10.2). This nonlinear transformation is defined implicitly by a Gaussian kernel with $\langle \Phi(x), \Phi(u) \rangle_{\mathcal{Z}} = \kappa(x, u)$. The key idea behind SVC is to find the smallest sphere in the feature space which encloses the data images $\{\Phi(x_1), \ldots, \Phi(x_n)\}$. That is, we aim to solve the minimization problem

$$\min_{\mathbf{a} \in \mathcal{Z}, R} R^2 \,, \text{ subject to } \|\Phi(x_j) - \mathbf{a}\|_{\mathcal{Z}}^2 \le R^2 \,, \forall j \,, \tag{10.17}$$

where R is the radius of an enclosing sphere in $\mathcal{Z}$. The Lagrangian is introduced to solve the above optimization problem. Let

$$L := R^2 - \sum_{j=1}^{n} (R^2 - \|\Phi(x_j) - \mathbf{a}\|^2) \beta_j \,, \tag{10.18}$$

where $\beta_j \ge 0$ are the Lagrange multipliers. By differentiating L with respect to the primal variables R and $\mathbf{a}$ respectively and setting the derivatives equal to zero, we have

$$\sum_{j}^{n} \beta_j = 1 \text{ and } \mathbf{a} = \sum_{j}^{n} \beta_j \Phi(x_j) \,. \tag{10.19}$$

Moreover, the corresponding Karush–Kuhn–Tucker complementarity conditions are

$$(R^2 - \|\Phi(x_j) - \mathbf{a}\|^2) \beta_j = 0, \forall j \,. \tag{10.20}$$

Combining (10.19) and (10.20), we can eliminate the primal variables R and $\mathbf{a}$ and get the following dual problem:

$$\max_{\beta} W(\beta) := \sum_{j=1}^{n} \beta_j \|\Phi(x_j) - \bar{\Phi}\|_{\mathcal{Z}}^2 \tag{10.21}$$

$$\text{s.t. } \bar{\Phi} := \sum_{j} \beta_j \Phi(x_j), \quad \sum_{j} \beta_j = 1 \text{ and } \beta_j \ge 0 \,.$$

That is, the SVC algorithm aims to find a weighting scheme β so that the weighted data spread $W(\beta)$ is as wide as possible.

As R is the radius of the enclosing sphere, corresponding pre-images of the enclosing sphere consist of points $\mathcal{C} := \{x : \|\Phi(x) - \bar{\Phi}\|_{\mathcal{Z}}^2 = R^2\}$. For $x \in \mathcal{C}$ we have

$$\|\Phi(x) - \bar{\Phi}\|_{\mathcal{Z}}^2 = \kappa(x,x) - 2\sum_{j=1}^{n} \beta_j \kappa(x_j, x) + \sum_{j,j'=1}^{n} \beta_j \beta_{j'} \kappa(x_j, x_{j'}) = R^2 .$$

Or equivalently

$$\mathcal{C} := \{x : \sum_{j=1}^{n} \beta_j \kappa(x_j, x) = \rho\} , \tag{10.22}$$

where $\rho = (\kappa(0,0) + \sum_{j,j'=1}^{n} \beta_j \beta_{j'} \kappa(x_j, x_{j'}) - R^2)/2$. When the enclosing sphere is mapped back to the data space $\mathcal{X}$, it forms a set of probability contours. These contours are used as cluster boundaries, and data points inside each contour are assigned to the same cluster. The SVC forms contours via kernel mixture (10.22), with the mixing coefficients β_j being solved from (10.21). Note that $\bar{\Phi}$ is a weighted centroid in the feature space and $\sum_{j=1}^{n} \beta_j \|\Phi(x_j) - \bar{\Phi}\|_{\mathcal{Z}}^2$ can be regarded as a weighted measure of data dispersion in the feature space. In other words, the SVC algorithm finds mixing coefficients to make the data dispersion as large as possible in the feature space $\mathcal{Z}$, while it draws the kernel mixture contours in the original data space $\mathcal{X}$ to form clusters. The set $\mathcal{C}$ defines the cluster boundaries. Data points lying on the boundaries are called support vectors. Note that the nonlinear transformation Φ is implicitly defined by a Gaussian kernel, $\kappa(x_j, x_{j'}) = e^{-q\|x_j - x_{j'}\|^2}$, $q > 0$. (The normalizing constant for κ is not relevant to cluster analysis and is dropped for simplicity.) A larger value of q corresponds to a smaller window width and leads to more clusters in the analysis.

Unlike the k-means algorithm, where the number of clusters k must be prescribed by the user, the window width in SVC can vary continuously, resulting in hierarchical clusters. The number of clusters depends on the window width of the Gaussian kernel. Decreasing the width leads to more clusters. Also, in contrast with the k-means procedure, no initial centroids are required as the algorithm input. Therefore, a deterministic result, independent from initial conditions, can be expected. SVC also has the ability to deal with outliers by employing slack variables. It allows some data points to remain outside the enclosing sphere in the feature space. This is same as the "soft margin" idea in support vector machines. With the introduction of slack variables, the optimization problem becomes

$$\min_{\mathbf{a}, R, \xi} R^2 + C\sum_{j=1}^{n} \xi_j , \text{ subject to } \|\Phi(x_j) - \mathbf{a}\|_{\mathcal{Z}}^2 \le R^2 + \xi_j , \ \xi_j \ge 0 , \ \forall j . \tag{10.23}$$

It is straightforward to derive the corresponding dual problem:

$$\max_{\beta} W(\beta) := \sum_{j=1}^{n} \beta_j \|\Phi(x_j) - \bar{\Phi}\|_{\mathcal{Z}}^2 \tag{10.24}$$

$$\text{s.t.} \bar{\Phi} := \sum_{j} \beta_j \Phi(x_j) , \ \sum_{i} \beta_j = 1 \text{ and } 0 \le \beta_j \le C .$$

As $\langle \Phi(x_j), \Phi(x_{j'}) \rangle_{\mathcal{Z}} = \kappa(x_j, x_{j'})$, the dual problem can be rewritten as a simple quadratic programming problem:

$$\max_{\beta} W(\beta) := -\sum_j \sum_{j'} \beta_j \beta_{j'} \kappa(x_j, x_{j'}) \tag{10.25}$$

$$\text{s.t.} \sum_j \beta_j = 1 \text{ and } 0 \le \beta_j \le C\,. \tag{10.26}$$

Solutions for β's are not unique, unless the kernel matrix $\mathbb{K} = [\kappa(x_j, x_{j'})]$ is of full rank. In an optimal solution to problem (10.25), if $0 < \beta_j < C$, then $\xi_j = 0$ and the corresponding data point x_j and its image $\Phi(x_j)$ lie, respectively, on the cluster boundaries and the surface of the sphere in $\mathcal{Z}$. Such a point is called a *support vector* (SV). The image of a point x_j with $\xi_j > 0$ lies outside the sphere. This implies that the corresponding $\beta_j = C$. Such an x_j is called a *bounded support vector* (BSV). Data points can be classified into three types: SVs lie on the cluster boundaries, BSVs are outside the boundaries, and the other points lie inside clusters. Since $0 \le \beta_j \le C$ and $\sum_{j=1}^n \beta_j = 1$, there is no BSV when $C \ge 1$. Moreover, $1/(nC)$ is an upper bound on the fraction of BSVs.

In three-dimensional space, we can easily visualize the clusters once the boundaries are drawn. However, in a data space with more dimensions, it is hard to picture and determine which data points are inside a specific cluster. Thus, we need an algorithm for cluster assignment. Ben-Hur et al. (2001) introduced the *adjacency matrix* for cluster assignment. Let $R(y) := \|\Phi(y) - \bar{\Phi}\|_{\mathcal{Z}}$ be the feature distance of $\Phi(y)$ to the data centroid $\bar{\Phi}$. Denote the adjacency matrix by $A = [A_{jj'}]$, where $A_{jj'} = 1$ indicates that the pair (j, j') is in the same cluster and $A_{jj'} = 0$ otherwise. We need only the upper triangular part of the A matrix. For $j < j'$

$$A_{jj'} = \begin{cases} 1: & \text{if } R(y) \le R, \forall y \text{ on the line segment connecting } x_j \text{ and } x_{j'}\,, \\ 0: & \text{otherwise}\,. \end{cases}$$

This definition is based on a geometric observation by Ben-Hur et al. For a given pair of data points, say x_j and $x_{j'}$, belonging to different clusters, any path that connects them must exit from the enclosing sphere. Therefore, such a path contains a segment of points y such that $R(y) > R$. Checking all points on a line segment is impossible. In practice, Ben-Hur et al., suggest that 10–20 uniformly distributed points should be used to check whether $R(y) > R$ or not. Once the adjacency matrix A is formed, the clusters can be defined as the connected components of the graph induced by A. This procedure will leave the BSVs as outliers. One can either assign them to the nearest clusters or leave them alone. We will illustrate an example to show how the SVC works and the effect of the window width of the Gaussian kernel.

Example 9 We synthesize 200 points in R^3 space, among which 80 points are in the upper hemisphere of the ball with radius 1.6 and with its center at the origin. The other 120 points are generated from three areas of the XY plane and then mapped into the lower hemisphere of the ball. We vary the parameter q from 1 to 7 and the number **9**

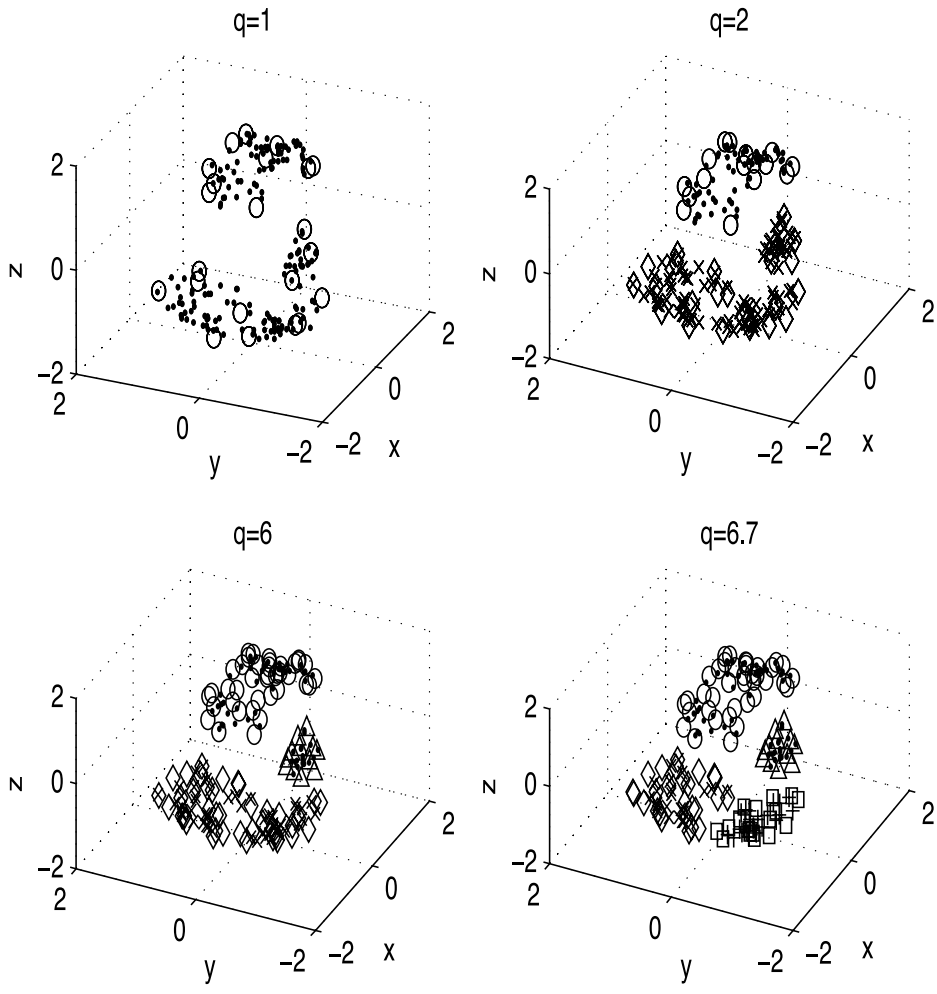


Figure 10.8. 200 points in a three-dimensional space

of resulting clusters changes from one to four. When $q = 1$, all 200 points are in one cluster, and when $q = 6.7$, we have four clusters, which is consistent with the way data is generated. The results are depicted in Fig. 10.8. Cluster membership is represented by different symbols; the circles, diamonds, squares and triangles indicate support vectors.

References

Aizerman, M.A., Braverman, E.M. and Rozoner, L.I. (1964). Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25:821–837.

Akaho, S. (2001). A kernel method for canonical correlation analysis. In: *International Meeting of Psychometric Society (IMPS2001), 15–19 July 2001, Osaka, Japan.*

Alpaydin, E. (2004). *Introduction to Machine Learning*. MIT Press, Cambridge.

Aronszajn, N. (1950). Theory of reproducing kernels. *Transactions of the American Mathematical Society*, 68:337–404.

Bach, F. and Jordan, M.I. (2002). Kernel independent component analysis. *Journal of Machine Learning Research*, 3:1–48.

Ben-Hur, A., Horn, D., Siegelmann, H.T. and Vapnik, V. (2001). Support vector clustering. *Journal of Machine Learning Research*, 2:125–137.

Berlinet, A. and Thomas-Agnan, C. (2004). *Reproducing Kernel Hilbert Spaces in Probability and Statistics*. Kluwer, Boston, MA.

Boser, B.E., Guyon, I.M. and Vapnik, V.N. (1992). A training algorithm for optimal margin classifiers. In: Valient, L. and Warmuth, M. (eds) *Proceedings of the 5th Annual ACM Workshop on Computational Learning Theory*. ACM Press, Pittsburgh, PA, 5:144–152.

Hardoon, D.R., Szedmak, S. and Shawe-Taylor, J. (2004). Canonical correlation analysis: An overview with application to learning methods. *Neural Computation*, 16(12): 2639–2664.

Hein, M. and Bousquet, O. (2004). Kernels, associated structures and generalizations. Technical report, Max Planck Institute for Biological Cybernetics, Germany. http://www.kyb.tuebingen.mpg.de/techreports.html.

Hotelling, H. (1936). Relations between two sets of variates. *Biometrika*, 28:321–377.

Lee, Y.J., Hsieh, W.F. and Huang, C.M. (2005). ϵ-SSVR: A smooth support vector machine for ϵ-insensitive regression. *IEEE Transactions on Knowledge and Data Engineering*, 17(5): 678–685.

Lee, Y.J. and Huang, S.Y. (2007). Reduced support vector machines: a statistical theory. *IEEE Transactions on Neural Networks*, 18:1–13.

Lee, Y.J. and Mangasarian, O.L. (2001a). RSVM: Reduced support vector machines. In: Kumar, V. and Grossman, R. (eds) *Proceedings of the First SIAM International Conference on Data Mining*. SIAM, Philadephia, PA.

Lee, Y.J. and Mangasarian, O.L. (2001b). SSVM: A smooth support vector machine for classification. *Computational Optimization and Applications*, 20(1): 5–22.

MacQueen, J.B. (1967). Some methods for classification and analysis of multivariate observations. In: Le Cam, L. and Neyman, J. (eds) *Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability*. University of California Press, Berkeley, CA, 1:281–297.

Mardia, K.V., Kent, J.T. and Bibby, J.M. (1979). *Multivariate Analysis*. Probability and Mathematical Statistics: A Series of Monographs and Textbooks. Academic, New York.

Schölkopf, B., Burges, C. and Smola, A. (1999). Kernel principal component analysis. *Advances in Kernel Methods – Support Vector Learning*, 327–352. MIT Press, Cambridge, MA.

Schölkopf, B., Smola, A. and Müller, K. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5): 1299–1319.

Vapnik, V.N. (1995). *The Nature of Statistical Learning Theory*. Springer-Verlag, New York.